

FAKULTET ELEKTROTEHNIKE I RAČUNARSTVA
SVEUČILIŠTE U ZAGREBU

High-speed and high-ratio referential genome compression

Stella Kiš, Natali Lazarić

Zagreb, 2026.

Sadržaj

1. High-speed and high-ratio referential genome compression - opis algoritma	3
2. Algoritam kroz korake	5
2.1 Učitavanje FASTA datoteke.....	5
2.2 Spajanje svih redaka u jednu cjelinu	5
2.3 Pronalaženje malih slova i stvaranje sekvence L1	5
2.4 Uklanjanje znakova N i stvaranje sekvence L2.....	6
2.5 Uklanjanje ostalih znakova i stvaranje sekvence L3	6
2.6 Pretvorba sekvence u numerički oblik.....	6
2.7 Generiranje k-torki	7
2.8 Izgradnja hash tablice nad k-torkama	7
2.9 Pronalaženje podudaranja između referentne i ciljane sekvence.....	8
2.10 Dodatno kodiranje podataka	8
2.11 Zapis rezultata kompresije u izlazne datoteke.....	9
2.12 Kompresija izlaznih datoteka pomoću 7zip alata	9
2.13 Ekstrakcija komprimirane arhive	10
2.14 Dekodiranje delta i RLE zapisa.....	10
2.15 Učitavanje podataka za dekompresiju.....	10
2.16 Rekonstrukcija sekvence L3, L2 i L1	11
2.17 Vraćanje malih slova i rekonstrukcija originalne sekvence	11
2.18 Validacija rekonstruirane sekvence.....	11
2.19 Zapis rekonstruirane FASTA datoteke	12
2.20 Završni rezultat algoritma.....	12
3. Rezultati testiranja	13
3.1 Skupovi podataka Escherichia coli	13
3.2 Skup kromosoma čovjeka	14
3.3 Utrošak memorije.....	15
4. Zaključak.....	16
5. Literatura.....	17

1. High-speed and high-ratio referential genome compression - opis algoritma

Razvoj tehnologija sekvenciranja doveo je do vrlo brzog rasta količine genomskih podataka koje je potrebno pohraniti, obrađivati i prenositi. To posebno dolazi do izražaja kod organizama za koje postoji velik broj međusobno vrlo sličnih genoma, kao što su različiti sojevi bakterija ili genomi različitih ljudi. U takvim slučajevima spremanje svakog genoma kao zasebne pune sekvence nije učinkovito, jer se velik dio informacije ponavlja. Zbog toga se javlja potreba za metodama kompresije koje iskorištavaju sličnost između genoma i tako značajno smanjuju veličinu zapisa.

HiRGC je algoritam za referencijsku kompresiju genoma bez gubitka podataka. Osnovna ideja takvog pristupa jest da se jedan genom uzima kao referenca, a drugi genom se ne zapisuje u cijelosti, nego se opisuje preko podudaranja s referencom i preko razlika koje se ne mogu izravno preslikati. Time se iskorištava činjenica da su srodni genomi u velikoj mjeri jednaki, dok se razlikuju samo u manjem broju pozicija, kraćih umetaka, brisanja ili zamjena. Cilj algoritma nije samo postići visok omjer kompresije, nego i omogućiti dovoljno brzo sažimanje i dekompresiju kako bi metoda bila praktična za rad nad velikim skupovima podataka.

Problem koji HiRGC rješava može se opisati ovako: za zadani referentni genom i zadani ciljni genom potrebno je pronaći način da se ciljni genom predstavi što kraćim zapisom, ali tako da se tijekom dekompresije može potpuno točno rekonstruirati izvorni sadržaj. Poseban izazov je u tome što se kao ulaz često koristi FASTA zapis, koji osim samih nukleotidnih znakova sadrži i pomoćne informacije poput zaglavlja i prijeloma redaka. Takvi podaci nisu dio same biološke sekvence, ali ih je također potrebno pravilno obraditi kako bi se sačuvala potpuna informacija o ulazu. HiRGC zato najprije odvaja stvarnu sekvencu od pomoćnih dijelova datoteke, a zatim glavnu obradu provodi nad "pročišćenim" nizom baza.

Jedna od važnih značajki algoritma je pretvorba nukleotida u vrlo kompaktan interni prikaz. Budući da osnovnu DNK sekvencu čine četiri glavne baze, moguće ih je kodirati s po 2 bita. Na taj se način ulazna sekvenca prevodi u numerički oblik koji zauzima manje prostora i pogodniji je za brzo pretraživanje podudaranja. Nakon toga se nad referentnom sekvencom gradi pomoćna struktura podataka, konkretno hash tablica, koja omogućuje brzo pronalaženje kandidata za podudaranja između reference i ciljne sekvence.

Središnji dio HiRGC algoritma temelji se na pohlepnoj strategiji traženja podnizova koji se poklapaju s referencom. Algoritam prolazi kroz ciljnu sekvencu i u svakom koraku pokušava pronaći što korisnije podudaranje u referentnom genomu. Kada pronade dovoljno dobro poklapanje, umjesto samih znakova zapisuje podatke koji opisuju gdje

se to podudaranje nalazi u referenci i kolike je duljine. Dijelovi koji se ne mogu učinkovito predstaviti preko reference zapisuju se kao nepodudarni simboli ili kratke razlike. Na taj se način dobiva kombinacija referenci na već postojeće segmente i izravno zapisanih odstupanja, što omogućuje znatno manju završnu veličinu datoteke.

Dekompresija radi obrnutim postupkom. Iz komprimiranog zapisa čitaju se informacije o podudaranjima i razlikama, a zatim se uz pomoć referentnog genoma rekonstruira izvorna ciljna sekvenca. Budući da je riječ o kompresiji bez gubitaka, rekonstruirani genom mora biti identičan izvornom ulazu. Upravo je to ključno svojstvo u bioinformatici, gdje i najmanja promjena u sekvenci može imati velik utjecaj na daljnju analizu.

Važnost HiRGC pristupa je u tome što pokušava istodobno zadovoljiti dva zahtjeva koja su često u suprotnosti: vrlo dobar stupanj kompresije i visoku brzinu izvođenja. U radu je algoritam predstavljen kao rješenje koje iskorištava sličnost među genomima, pojednostavljuje obradu sekvence pomoću 2-bitnog kodiranja i ubrzava traženje podudaranja uporabom hash tablice i pohlepne strategije pretraživanja. Zbog toga je prikladan za scenarije u kojima se obrađuju velike kolekcije sličnih genoma, poput skupova bakterijskih genoma ili ljudskih kromosoma.

2. Algoritam kroz korake

2.1 Učitavanje FASTA datoteke

Prvi korak je čitanje ulazne FASTA datoteke. Iz datoteke se izdvaja identifikator sekvence, odnosno tekst iza znaka >, dok se svi ostali redci koji predstavljaju dijelove nukleotidne sekvence spremaju u zasebno polje. Uz to se za svaki redak pamti i njegova duljina. Time se ne čuva samo sama sekvenca, nego i informacija o izvornom rasporedu redaka, što je važno za kasniju potpunu rekonstrukciju ulazne datoteke. Takav pristup omogućuje odvajanje pomoćnih informacija od stvarne sekvence koja se dalje obrađuje.

U implementaciji to radi funkcija `read_fasta` koja otvara FASTA datoteku, čita identifikator, sprema svaki redak sekvence u vektor `sequences` i pamti duljinu svakog retka u `seq_len`.

2.2 Spajanje svih redaka u jednu cjelinu

Nakon učitavanja FASTA datoteke, svi redci sekvence spajaju se u jedan dugi niz znakova. Time se dobiva jedinstvena sekvenca pogodna za daljnju obradu, bez prekida nastalih zbog formatiranja u datoteci.

U kodu je to ostvareno funkcijom `join_sequences`, koja sve elemente iz vektora `sequences` spaja u jednu string varijablu.

2.3 Pronalaženje malih slova i stvaranje sekvence L1

Sljedeći korak pretprocesiranja odnosi se na mala slova u sekvenci. U genomskim zapisima mala slova često nose dodatnu informaciju, pa ih nije dovoljno samo pretvoriti u velika slova i zanemariti njihov izvorni položaj. Zbog toga se najprije pronalaze svi intervali uzastopnih malih slova. Za svaki takav interval bilježi se početna pozicija i duljina.

Nakon što su ti intervali zabilježeni, cijela sekvenca pretvara se u velika slova, čime se dobiva nova sekvenca L1. Na taj način sadržaj sekvence se pojednostavljuje za daljnju obradu, ali se i dalje zadržava informacija o tome gdje su se nalazila mala slova, što omogućuje kasniju rekonstrukciju originalnog zapisa.

U implementaciji to obavlja funkcija `preprocess_sequence` kroz sljedeća polja:

- `low_pos` – početne pozicije intervala malih slova
- `low_len` – duljine tih intervala
- `L1` – sekvenca nakon pretvorbe svih znakova u velika slova

2.4 Uklanjanje znakova N i stvaranje sekvence L2

Nakon dobivanja sekvence L1, obrađuju se znakovi N. Ovaj znak obično označava nepoznatu ili nedefiniranu bazu i ne ulazi u osnovni skup nukleotida koji se dalje koristi. Kako bi se sekvenca dodatno pojednostavila, ponovno se traže intervali uzastopnih pojavljivanja znaka N.

Za svaki takav interval bilježi se njegova početna pozicija i duljina, nakon čega se svi znakovi N uklanjaju iz sekvence. Rezultat tog postupka je nova sekvenca L2 koja više ne sadrži nepoznate baze, ali su sve potrebne informacije o njihovim pozicijama sačuvane za kasniju upotrebu.

U implementaciji su za to predviđena polja:

- N_pos – početne pozicije intervala znaka N
- N_len – duljine tih intervala
- L2 – sekvenca iz koje su svi N znakovi uklonjeni

2.5 Uklanjanje ostalih znakova i stvaranje sekvence L3

Nakon uklanjanja znakova N, u sekvenci još uvijek mogu postojati znakovi koji ne pripadaju standardnim bazama A, C, G i T. Takvi znakovi se u ovoj fazi izdvajaju iz glavne sekvence, ali se njihov položaj i vrijednost pritom ne gube.

Za svaki takav znak bilježi se njegova pozicija i sam znak, a zatim se uklanja iz sekvence. Time se dobiva sekvenca L3 koja sadrži isključivo četiri osnovne baze. Ovakva sekvenca je pogodna za daljnju obradu jer više ne sadrži iznimke koje bi komplicirale algoritam.

U implementaciji se to sprema u:

- oth_pos – pozicije ostalih znakova
- oth_ch – sami znakovi koji nisu A, C, G ili T
- L3 – završna pročišćena sekvenca

2.6 Pretvorba sekvence u numerički oblik

Nakon što je dobivena sekvenca L3, svaki nukleotid prevodi se u brojčanu vrijednost. U implementaciji se koristi sljedeće preslikavanje: A → 0, C → 1, G → 2, T → 3.

Na taj način sekvenca više nije predstavljena kao niz znakova, nego kao niz cijelih brojeva. Ovakav oblik zapisa jednostavniji je za daljnju obradu jer omogućuje brže usporedbe i lakše formiranje podnizova.

U kodu je ovaj korak implementiran funkcijom `to_binary`, koja iz sekvence L3 stvara vektor `encoded`.

2.7 Generiranje k-torki

Nakon numeričkog kodiranja, iz dobivene sekvence generiraju se uzastopne k-torke. K-torka predstavlja podniz od k susjednih elemenata numeričke sekvence. Za svaku takvu k-torku računa se jedna numerička vrijednost i pamti se njezina početna pozicija.

Ovaj korak je važan jer omogućuje rad s kraćim segmentima sekvence umjesto s pojedinačnim znakovima, što pojednostavljuje daljnju obradu i usporedbu sekvenci.

U implementaciji se koristi *rolling hash* pristup temeljen na base-4 kodiranju. Time se izbjegava ponovno računanje cijele k-torke za svaku poziciju, što značajno smanjuje vremensku složenost.

Svaka k-torka predstavljena je hash vrijednošću (base-4 kodiranje) te početnom pozicijom u sekvenci.

Rolling hash se održava tako da se postojeća vrijednost pomiče (*shift*), uklanja se najstariji element (nukleotid) te se dodaje novi.

2.8 Izgradnja hash tablice nad k-torkama

Nakon generiranja k-torki, sljedeći korak je njihovo organiziranje u strukturu koja omogućuje brzo i učinkovito pretraživanje. U tu svrhu koristi se hash tablica, koja omogućuje grupiranje k-torki prema njihovim vrijednostima i brži pristup podacima. Umjesto da se svaka k-torka traži prolaskom kroz cijelu sekvencu, ovakav pristup omogućuje izravan pristup svim njezinim pojavljivanjima.

Svaka k-torka je predstavljena kao 64-bitna vrijednost dobivena base-4 kodiranjem (svaka baza A, C, G, T mapira se na 2 bita). Na temelju te vrijednosti računa se hash indeks pomoću bit-maske:

- indeks se dobiva operacijom $\text{value} \& (s - 1)$, gdje je s veličina hash tablice i mora biti potencija broja 2

Na taj način k-torke se raspoređuju u hash „bucket-e“, pri čemu više k-torki može pripadati istom indeksu.

Hash struktura sastoji se od dva glavna polja:

- **h** – za svaki hash indeks čuva indeks prve k-torke u tom bucketu
- **p** – za svaku k-torku čuva pokazivač na prethodnu k-torku u istom bucketu

Prilikom umetanja k-torki, svaka nova k-torka se dodaje na početak odgovarajućeg bucket lanca. Time se formira jednostruka povezana struktura unutar svakog hash indeksa, gdje se sve k-torke s istim hashom povezuju preko niza indeksa u polju **p**.

Ovakva organizacija omogućuje da se sve k-torke koje potencijalno odgovaraju istoj vrijednosti mogu dohvatiti u konstantnom vremenu po bucketu, a zatim precizno provjeriti usporedbom stvarnih vrijednosti.

U implementaciji ovaj postupak provodi funkcija koja prima vektor vrijednosti k-torki i veličinu hash tablice te vraća strukturu koja omogućuje brzo pretraživanje svih pojavljivanja pojedine k-torke tijekom faze greedy usporedbe sekvenci.

2.9 Pronalaženje podudaranja između referentne i ciljane sekvence

Nakon izgradnje hash tablice nad k-torkama referentne sekvence, cilj je pronaći dijelove ciljane sekvence koji se mogu opisati pomoću dijelova referentne sekvence. Time se ne sprema cijela ciljana sekvenca, nego samo informacije o tome gdje se određeni dio nalazi u referentnoj sekvenci i kolika je njegova duljina.

Algoritam prolazi kroz ciljanu sekvencu i za svaku poziciju određuje vrijednost trenutne k-torke. Umjesto ponovnog računanja za svaku poziciju, koristi se klizni prozor (*rolling hash*), čime se vrijednost k-torke ažurira u konstantnom vremenu.

Zatim se pomoću hash tablice pronalaze kandidati iz referentne sekvence koji imaju istu hash vrijednost. Budući da različite k-torke mogu imati istu hash vrijednost, za svakog kandidata dodatno se provjerava stvarna jednakost k-torki ciljane sekvence. Ako se podudaraju, podudaranje se dalje proširuje usporedbom znakova ciljane i referentne sekvence sve dok se oni poklapaju.

Za svaku poziciju odabire se najdulje pronađeno podudaranje (pohlepni pristup). Ako se pronađe podudaranje duljine barem k , ono se sprema kao zapis tipa Match. Ako između prethodnog i trenutnog podudaranja postoji dio sekvence koji nije pronađen u referentnom genomu, taj se dio sprema kao Mismatch.

Nakon pronalaska podudaranja, algoritam preskače cijeli podudarni segment i nastavlja pretraživanje od sljedeće neobrađene pozicije.

U implementaciji ovaj dio obavlja funkcija `greedy_matching`, koja stvara:

- `matches` – popis pronađenih podudaranja
- `mismatches` – dijelove ciljane sekvence koji se ne mogu opisati referencom

2.10 Dodatno kodiranje podataka

Nakon pronalaska podudaranja provodi se dodatno smanjenje količine podataka primjenom više jednostavnih metoda kodiranja. Budući da su mnoge pozicije zapisane rastućim redoslijedom, koristi se delta kodiranje, pri čemu prva vrijednost ostaje nepromijenjena, a svaka sljedeća zapisuje se kao razlika u odnosu na prethodnu. Time

se dobivaju manji brojevi, pogodniji za daljnju kompresiju. U implementaciji se to provodi funkcijom `delta_encode` nad pozicijama podudaranja, pozicijama u referentnoj sekvenci te pozicijama malih slova, znakova N i ostalih znakova.

Kako bi se mogla obnoviti izvorna struktura FASTA datoteke, dodatno se kodiraju duljine redaka. Budući da se iste duljine često ponavljaju, koristi se RLE kodiranje koje sprema vrijednost i broj njezinih uzastopnih pojavljivanja. Time se izbjegava višestruko spremanje istih podataka. U implementaciji se ovaj postupak provodi funkcijom `rle_encode`, koja iz vektora `seq_len` stvara niz vrijednosti i njihovih ponavljanja.

Osim toga, znakovi koji ne pripadaju skupu A, C, G i T izdvajaju se i kodiraju pomoću pomoćne abecede. Najprije se formira skup svih različitih takvih znakova, a zatim se svaki znak zamjenjuje indeksom u toj abecedi. Na taj način se i ovi znakovi zapisuju u kompaktnijem numeričkom obliku. U implementaciji se koriste funkcije `build_other_alphabet` i `encode_other_chars`, a dobiveni podaci spremaju se kako bi se tijekom dekompresije mogli vratiti u izvorni oblik.

2.11 Zapis rezultata kompresije u izlazne datoteke

Nakon što su pronađena podudaranja, nepodudaranja i pomoćni podaci, rezultati se zapisuju u tri odvojene tekstualne datoteke funkcijom `write_all_outputs`. Na taj način se glavni podaci kompresije odvajaju od pomoćnih informacija potrebnih za rekonstrukciju.

U datoteku `matches.txt` zapisuju se podudaranja. Pritom se početne pozicije i pozicije u referentnoj sekvenci spremaju delta kodirano, dok se duljine podudaranja spremaju izravno.

U datoteku `mismatches.txt` zapisuju se dijelovi sekvence koji nisu pronađeni u referentnom genomu. Za svaki takav dio sprema se njegova početna pozicija i stvarni niz znakova.

U datoteku `auxiliary.txt` zapisuju se pomoćni podaci potrebni za rekonstrukciju izvorne FASTA datoteke. To uključuje identifikator sekvence, duljine redaka, informacije o malim slovima, intervale znakova N te podatke o ostalim znakovima.

2.12 Kompresija izlaznih datoteka pomoću 7zip alata

Nakon što su svi potrebni podaci zapisani u tekstualne datoteke, one se dodatno komprimiraju vanjskim alatom 7zip. Time se tri pomoćne datoteke pakiraju u jednu arhivu, čime se dobiva konačni komprimirani zapis.

U implementaciji se ovaj korak obavlja funkcijom `compress_with_7zip`. Ona pomoću naredbe sustava poziva 7zip i stvara arhivu `compressed_output.7z` iz datoteka: `matches.txt`, `mismatches.txt` i `auxiliary.txt`

2.13 Ekstrakcija komprimirane arhive

Postupak dekompresije započinje izdvajanjem datoteka iz prethodno stvorene 7zip arhive. Time se ponovno dobivaju datoteke koje sadrže podudaranja, nepodudaranja i pomoćne podatke potrebne za rekonstrukciju sekvence.

U implementaciji se ovaj korak obavlja funkcijom `extract_with_7zip`. Ona poziva 7zip naredbu za raspakiravanje arhive `compressed_output.7z` u direktorij `extracted`.

2.14 Dekodiranje delta i RLE zapisa

Nakon učitavanja raspakiranih datoteka potrebno je vratiti podatke u njihov izvorni oblik. Vrijednosti koje su tijekom kompresije bile delta kodirane ponovno se pretvaraju u apsolutne pozicije. To se radi tako da se svaka sljedeća vrijednost dobije zbrajanjem prethodne rekonstruirane vrijednosti i trenutne delta vrijednosti.

Osim toga, duljine redaka FASTA datoteke koje su bile zapisane RLE postupkom ponovno se proširuju u izvorni niz duljina.

U implementaciji se koriste funkcije:

- `delta_decode` – vraća delta kodirane vrijednosti u apsolutne pozicije
- `rle_decode` – obnavlja puni niz duljina redaka iz RLE zapisa

2.15 Učitavanje podataka za dekompresiju

Nakon raspakiravanja arhive, program učitava tri tekstualne datoteke. Iz datoteke `matches.txt` učitavaju se podudaranja, pri čemu se njihove pozicije najprije delta dekodiraju. Iz datoteke `mismatches.txt` učitavaju se dijelovi sekvence koji nisu bili opisani referentnim genomom. Iz datoteke `auxiliary.txt` učitavaju se svi pomoćni podaci potrebni za potpunu rekonstrukciju originalne FASTA datoteke.

U implementaciji se za to koriste funkcije:

- `read_matches` – učitava i dekodira podudaranja
- `read_mismatches` – učitava nepodudarne dijelove
- `read_auxiliary` – učitava identifikator, duljine redaka i pomoćne informacije

2.16 Rekonstrukcija sekvence L3, L2 i L1

Postupak rekonstrukcije započinje obnovom sekvence L3, odnosno pročišćene sekvence koja sadrži samo znakove A, C, G i T. Ona se dobiva kombiniranjem podataka o podudaranjima i nepodudaranjima. Dijelovi označeni kao nepodudaranja izravno se dodaju u rezultat, dok se podudarni dijelovi preuzimaju iz referentne sekvence na temelju spremljene pozicije i duljine. U implementaciji se ovaj korak obavlja funkcijom `reconstruct_L3`.

Nakon toga slijedi rekonstrukcija sekvence L2, u koju se vraćaju svi znakovi koji ne pripadaju osnovnom skupu nukleotida, a koji su tijekom pretprocesiranja bili uklonjeni. Za svaki takav znak koristi se njegova spremljena pozicija i vrijednost te se umeće natrag u sekvencu. Time se dobiva sekvenca prije uklanjanja ostalih znakova. Ovaj postupak implementiran je funkcijom `reconstruct_L2`.

U posljednjem koraku obnavlja se sekvenca L1 dodavanjem znakova N. Budući da su ti znakovi spremljeni kao intervali, za svaki interval poznata je početna pozicija i duljina pa se odgovarajući niz znakova umeće na predviđena mjesta. Rezultat je sekvenca koja sadrži sve znakove u obliku velikih slova. U implementaciji se ovaj korak obavlja funkcijom `reconstruct_L1`.

2.17 Vraćanje malih slova i rekonstrukcija originalne sekvence

Nakon što je obnovljena sekvenca L1, posljednji korak rekonstrukcije sadržaja je vraćanje malih slova. Tijekom pretprocesiranja mala slova nisu izgubljena, nego su njihovi intervali spremljeni u pomoćne podatke.

Za svaki interval malih slova program prolazi kroz odgovarajući dio sekvence i znakove pretvara natrag u mala slova. Time se dobiva rekonstruirana sekvenca koja sadržajno odgovara originalnoj sekvenci iz ulazne FASTA datoteke. U implementaciji se ovaj dio obavlja funkcijom `reconstruct_original`.

2.18 Validacija rekonstruirane sekvence

Nakon rekonstrukcije originalne sekvence provodi se provjera ispravnosti.

Rekonstruirana sekvenca uspoređuje se s izvornom sekvencom dobivenom na početku programa spajanjem redaka iz ulazne FASTA datoteke.

Ako su sekvence jednake, dekompresija se smatra uspješnom. Ako postoji razlika, program ispisuje prvu poziciju na kojoj se sekvence razlikuju te prikazuje originalni i rekonstruirani znak na toj poziciji. Time se olakšava pronalazak pogreške u postupku kompresije ili dekompresije.

2.19 Zapis rekonstruirane FASTA datoteke

Nakon uspješne rekonstrukcije sekvence, potrebno je ponovno zapisati FASTA datoteku u izvornom obliku. Osim same sekvence, koristi se i spremljeni identifikator te duljine redaka originalne datoteke.

Program najprije zapisuje znak > i identifikator sekvence, a zatim sekvencu dijeli na retke prema prethodno spremljenim duljinama. Time se ne obnavlja samo sadržaj sekvence, nego i izvorni raspored redaka FASTA datoteke.

U implementaciji se ovaj dio obavlja funkcijom `write_fasta`, koja stvara datoteku `reconstructed.fna`.

2.20 Završni rezultat algoritma

Završni rezultat kompresije je arhiva `compressed_output.7z`, koja sadrži sve podatke potrebne za obnovu ciljane FASTA datoteke. U arhivi se nalaze podaci o podudaranjima s referentnim genomom, nepodudarnim dijelovima sekvence i pomoćne informacije potrebne za vraćanje originalnog formata.

Završni rezultat dekompresije je datoteka `reconstructed.fna`. Ona se dobiva rekonstrukcijom svih prethodno uklonjenih ili kodiranih dijelova sekvence te ponovnim zapisom u FASTA format.

Uspješnost algoritma potvrđuje se usporedbom rekonstruirane sekvence s originalnom sekvencom. Ako su one jednake, znači da su kompresija i dekompresija provedene ispravno te da su svi potrebni podaci sačuvani.

3. Rezultati testiranja

U ovom poglavlju prikazani su rezultati testiranja implementiranog algoritma, pri čemu se analizira vrijeme izvođenja te utrošak memorije faze kompresije. Mjerenja su provedena kako bi se procijenila učinkovitost implementacije u odnosu na originalnu verziju algoritma. Također, mjerenja su provedena za različite skupove podataka kako bi se dobio uvid u ponašanje algoritma na sekvencama različite veličine i složenosti.

3.1 Skupovi podataka Escherichia coli

Ovo poglavlje sadrži prikaz rezultata testiranja implementiranog algoritma na stvarnim genomskim podacima bakterije Escherichia coli. Za razliku od klasičnog pristupa s jednim referentnim genomom, u ovom radu provedene su međusobne usporedbe između više različitih sojeva.

U testiranju su korišteni sljedeći genomi:

- Escherichia coli K-12 (GCA_000005845.2)
- Escherichia coli O157:H7 Sakai (GCA_000008865.2)
- Escherichia coli SMS-3-5 (GCA_000019645.1)
- Escherichia coli UTI89 (GCA_000013265.1)

Testiranje je provedeno između različitih parova genoma, pri čemu su uloge referentne i ciljne sekvence definirane unaprijed za svaki eksperiment. Tablica rezultata prikazuje usporedbu vremena izvođenja originalne i ostvarene implementacije, s ciljem evaluacije učinkovitosti ostvarene implementacije.

CILJANA SEKVENCA	REFERENTNA SEKVENCA	VRIJEME TRAJANJA ORIGINALNE IMPLEMENTACIJE	VRIJEME TRAJANJA OSTVARENE IMPLEMENTACIJE
Escherichia coli O157:H7 Sakai	Escherichia coli K-12	19.016 s	21.3909 s
Escherichia coli SMS-3-5	Escherichia coli K-12	21.5489 s	21.7513 s
Escherichia coli UTI89	Escherichia coli K-12	19.2165 s	21.4054 s
Escherichia coli O157:H7 Sakai	Escherichia coli SMS-3-5	20.2251 s	22.6421 s
Escherichia coli UTI89	Escherichia coli SMS-3-5	19.2192 s	21.5129 s
Escherichia coli K-12	Escherichia coli SMS-3-5	18.5159 s	21.8747 s

3.2 Skup kromosoma čovjeka

U ovom poglavlju prikazani su rezultati testiranja implementiranog algoritma na stvarnim genomskim podacima čovjeka (*Homo sapiens*). Za razliku od bakterijskih genoma koji su relativno mali, ovdje se radi o znatno većim i kompleksnijim sekvencama, što omogućuje dodatnu evaluaciju skalabilnosti i učinkovitosti algoritma.

U testiranju su korištene sekvence različitih ljudskih kromosoma iz skupa podataka GCA_000001405.29:

- *Homo sapiens* chromosome 19
- *Homo sapiens* chromosome 20
- *Homo sapiens* chromosome 21
- *Homo sapiens* chromosome 22

Testiranje je provedeno između različitih parova kromosoma istog čovjeka, pri čemu su uloge referentne i ciljne sekvence mijenjane između eksperimenata. Time je omogućena analiza performansi algoritma na većim i složenijim genomskim strukturama.

Tablica prikazuje usporedbu vremena izvođenja originalne i ostvarene implementacije. S obzirom na veću veličinu podataka, posebna pažnja posvećena je utjecaju optimizacija na vrijeme izvođenja te ukupnoj učinkovitosti algoritma u uvjetima rada s velikim genomskim sekvencama.

CILJANA SEKVENCA	REFERENTNA SEKVENCA	VRIJEME TRAJANJA ORIGINALNE IMPLEMENTACIJE	VRIJEME TRAJANJA OSTVARENE IMPLEMENTACIJE
<i>Homo sapiens</i> chromosome 20	<i>Homo sapiens</i> chromosome 19	2 min 27 s	3 min 12 s
<i>Homo sapiens</i> chromosome 21	<i>Homo sapiens</i> chromosome 19	1 min 23 s	2 min 11 s
<i>Homo sapiens</i> chromosome 22	<i>Homo sapiens</i> chromosome 19	1 min 43 s	2 min 45 s
<i>Homo sapiens</i> chromosome 19	<i>Homo sapiens</i> chromosome 22	2 min 12 s	2 min 40 s
<i>Homo sapiens</i> chromosome 20	<i>Homo sapiens</i> chromosome 22	1 min 54 s	2 min 14 s
<i>Homo sapiens</i> chromosome 21	<i>Homo sapiens</i> chromosome 22	1 min 16 s	1 min 32 s

3.3 Utrošak memorije

Tijekom testiranja mjerena je najveća zauzeta količina radne memorije tijekom izvođenja kompresije za originalnu implementaciju HiRGC algoritma i ostvarenu implementaciju. Mjerenje je provedeno korištenjem funkcije `getrusage()` nad glavnim procesom i pomoćnim procesima korištenima za 7zip kompresiju. Rezultati pokazuju da ostvarena implementacija koristi nešto više memorije u odnosu na originalnu implementaciju za sve ispitane skupove podataka. Kod bakterijskih genoma razlika u potrošnji memorije iznosi približno 200 MB, dok je kod ljudskih kromosoma povećanje izraženije te doseže oko 2–3 GB. Razlog povećane memorijske potrošnje ostvarene implementacije proizlazi iz dodatnih struktura podataka korištenih tijekom predobrade, pohrane pomoćnih informacija i rekonstrukcije sekvence. Unatoč tome, ukupna potrošnja memorije ostaje unutar očekivanih granica s obzirom na veličinu obrađivanih genomskih podataka i korištene hash strukture velikog kapaciteta.

CILJANA SEKVENCA	REFERENTNA SEKVENCA	UTROŠAK MEMORIJE ORIGINALNE IMPLEMENTACIJE	UTROŠAK MEMORIJE OSTVARENE IMPLEMENTACIJE
Escherichia coli O157:H7 Sakai	Escherichia coli K-12	8313.25 MB	8560.66 MB
Escherichia coli SMS-3-5	Escherichia coli K-12	8310.50 MB	8533.94 MB
Escherichia coli UTI89	Escherichia coli K-12	8309.75 MB	8533.42 MB
Homo sapiens chromosome 20	Homo sapiens chromosome 19	9584.75 MB	12403.9 MB
Homo sapiens chromosome 21	Homo sapiens chromosome 19	9400.50 MB	11971.1 MB
Homo sapiens chromosome 22	Homo sapiens chromosome 19	9394.50 MB	11972.3 MB

4. Zaključak

Iz prikazanih rezultata može se uočiti razlika u performansama između originalne i ostvarene implementacije na oba skupa podataka.

Kod skupa podataka *Escherichia coli* originalna implementacija u svim testovima ostvaruje nešto kraće vrijeme izvođenja. Vrijeme originalne implementacije kreće se približno od 18,52 s do 21,55 s, dok ostvarena implementacija postiže rezultate u rasponu od 21,39 s do 22,64 s. Razlike su relativno male i u većini slučajeva iznose nekoliko sekundi. Najmanja razlika iznosi oko 0,20 s, dok najveća doseže približno 3,36 s. Ovakvi rezultati pokazuju da ostvarena implementacija uspješno prati funkcionalnost originalnog algoritma, ali uz blago povećanje vremena izvođenja.

Kod skupa podataka *Homo sapiens* razlike postaju znatno izraženije. Originalna implementacija ostvaruje vremena izvođenja u rasponu od 1 min 16 s do 2 min 27 s, dok ostvarena implementacija zahtijeva od 1 min 32 s do 3 min 12 s. Razlike su u ovom slučaju značajnije te se kreću od približno 16 s do više od jedne minute. Povećanje vremena izvođenja posebno dolazi do izražaja zbog znatno većih i kompleksnijih genomskih sekvenci u odnosu na bakterijske podatke.

Razlog ovakvog ponašanja leži u razlikama između implementacija. Originalni kod je snažno optimiziran za brzinu izvođenja, koristi statički alocirane nizove, minimalan broj pomoćnih struktura i direktnu manipulaciju memorijom. S druge strane, ostvarena implementacija koristi moderniji i modularniji pristup temeljen na strukturama podataka poput vector, dodatnim koracima obrade (npr. višefazni preprocessing, delta i RLE kodiranje) te zapisivanju rezultata u više datoteka prije konačne kompresije.

Takav pristup poboljšava čitljivost, održivost i razumljivost koda, kao i mogućnost rekonstrukcije podataka, ali uvodi dodatni vremenski trošak. Taj trošak je relativno mali kod manjih genoma poput *Escherichia coli*, dok kod većih sekvenci poput *Homo sapiens* postaje znatno izraženiji.

Zaključno, ostvarena implementacija uspješno ispunjava funkcionalne zahtjeve i omogućuje točnu kompresiju i dekompresiju genomske sekvence, ali u pogledu vremena izvođenja ne nadmašuje originalnu implementaciju. Originalni algoritam ostaje učinkovitiji u smislu performansi, dok ostvarena implementacija predstavlja kompromis između performansi i kvalitete programskog rješenja.

5. Literatura

Liu et al. 2017. High-speed and high-ratio referential genome compression,
<https://academic.oup.com/bioinformatics/article/33/21/3364/3885699>

Originalna implementacija,
<https://github.com/yuansliu/HiRGC/tree/master>

Homo sapiens chromosome 19, GRCh38.p14 Primary Assembly,
https://www.ncbi.nlm.nih.gov/nuccore/NC_000019.10?report=fasta

Homo sapiens chromosome 20, GRCh38.p14 Primary Assembly,
https://www.ncbi.nlm.nih.gov/nuccore/NC_000020.11?report=fasta

Homo sapiens chromosome 21, GRCh38.p14 Primary Assembly,
https://www.ncbi.nlm.nih.gov/nuccore/NC_000021.9?report=fasta

Homo sapiens chromosome 22, GRCh38.p14 Primary Assembly,
https://www.ncbi.nlm.nih.gov/nuccore/NC_000022.11?report=fasta

Escherichia coli SMS-3-5,
https://www.ncbi.nlm.nih.gov/datasets/genome/GCF_000019645.1/

Escherichia coli str. K-12,
https://www.ncbi.nlm.nih.gov/datasets/genome/GCF_000005845.2/

Escherichia coli O157:H7 Sakai,
https://www.ncbi.nlm.nih.gov/datasets/genome/GCF_000008865.2/

Escherichia coli UTI89,
https://www.ncbi.nlm.nih.gov/datasets/genome/GCF_000013265.1/